

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA0706

COURSE OUTCOME COVERED

CO5:

CO5: Measure network performance using key metrics with simulation tools.

(BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I C KARTHIK REDDY(192425157)certify that this submission is my original workand that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

C KARTHIK REDDY

Pseudocode Writing Task

Algorithm Design for Network Monitoring

1. Throughput and Packet Loss Calculation

Problem Understanding

The network simulation provides the following values:

Total data received = 8000 bits

Time = 4 seconds

Total packets sent = 200 packets

Total packets received = 180 packets

The algorithm must calculate:

Throughput

Throughput=

Time

Total Data Received

Packet Loss

Packet Loss=

Packets Sent

Packets Sent–Packets Received×100

The network is classified as Efficient when:

Throughput > 1500 bps

Packet loss < 10%

Otherwise, it is classified as Inefficient.

Algorithm Design

BEGIN

SET total_data_received = 8000

SET time = 4

SET packets_sent = 200

SET packets_received = 180

IF time <= 0 THEN

 DISPLAY "Error: Time must be greater than zero"

 STOP

END IF

IF packets_sent <= 0 THEN

 DISPLAY "Error: Total packets sent must be greater than zero"

 STOP

END IF

IF packets_received < 0 OR packets_received > packets_sent THEN

 DISPLAY "Error: Invalid number of packets received"

 STOP

END IF

throughput = total_data_received / time

lost_packets = packets_sent - packets_received

packet_loss = (lost_packets / packets_sent) * 100

DISPLAY "NETWORK PERFORMANCE SUMMARY"

DISPLAY "Total Data Received: ", total_data_received, " bits"

DISPLAY "Time: ", time, " seconds"

DISPLAY "Packets Sent: ", packets_sent

DISPLAY "Packets Received: ", packets_received

DISPLAY "Packets Lost: ", lost_packets

DISPLAY "Throughput: ", throughput, " bps"

DISPLAY "Packet Loss: ", packet_loss, "%"

IF throughput > 1500 AND packet_loss < 10 THEN

 classification = "Efficient"

ELSE

 classification = "Inefficient"

END IF

DISPLAY "Network Classification: ", classification

IF classification = "Efficient" THEN

 DISPLAY "The network provides good throughput and acceptable packet loss."

ELSE

 DISPLAY "The network requires performance improvement."

END IF

END

Calculation

Throughput:

4

8000

=2000 bps

Packets lost:

$200 - 180 = 20$

Packet loss:

200

20

$\times 100 = 10\%$

Therefore:

Throughput = 2000 bps

Packet Loss = 10%

Throughput is greater than 1500 bps.

Packet loss is not less than 10%.

Hence, the network is:

Classification = Inefficient

Use of Control Structures

The algorithm uses:

IF conditions for input validation.

Arithmetic operations for metric calculation.

Conditional classification using AND.

STOP to prevent division-by-zero errors.

Completeness

The algorithm handles:

Invalid time.

Zero packets sent.

Invalid received-packet values.

Throughput calculation.

Packet loss calculation.

Network classification.

Complete performance summary.

Final Output

NETWORK PERFORMANCE SUMMARY

Total Data Received: 8000 bits

Time: 4 seconds

Packets Sent: 200

Packets Received: 180

Packets Lost: 20

Throughput: 2000 bps

Packet Loss: 10%

Network Classification: Inefficient

The network requires performance improvement.

Throughput & Packet Loss Algorithm

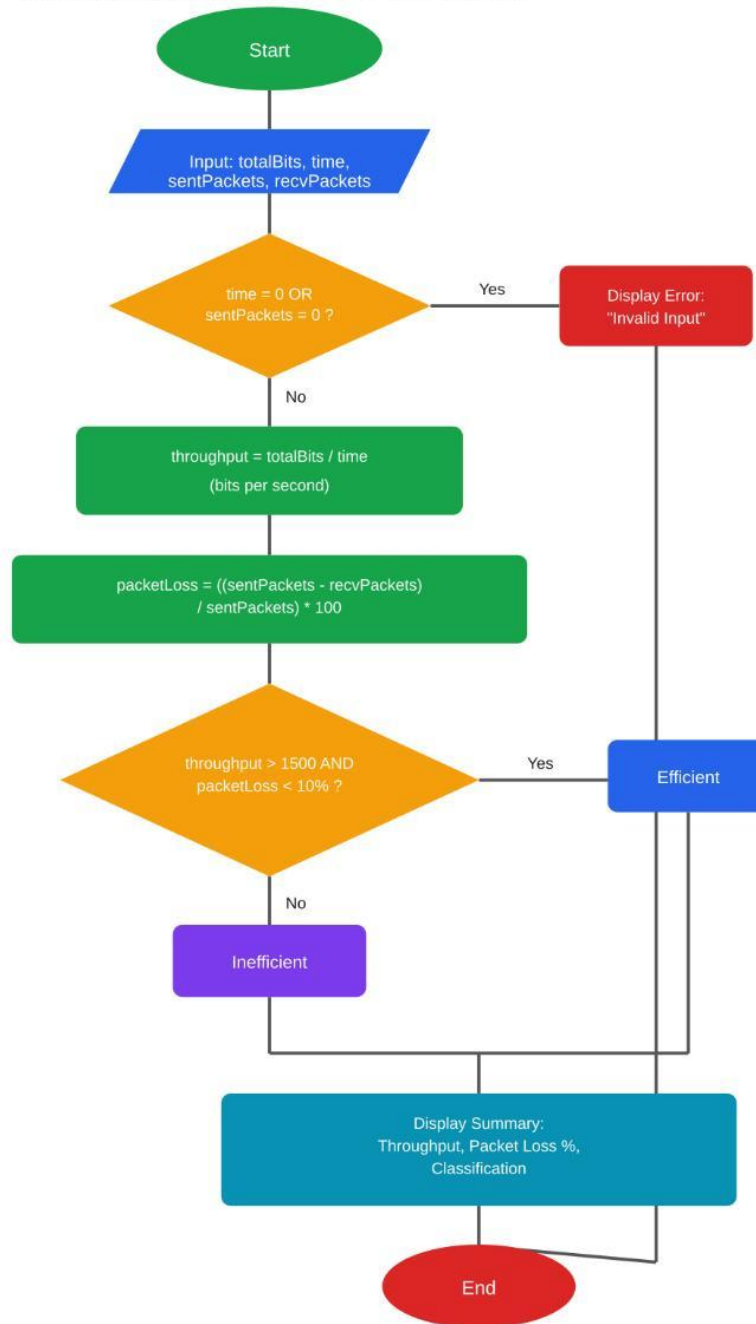


Figure 1: Flowchart — validation, calculation, classification, and summary output

2. Bandwidth Utilization Monitoring Across Branch Links

Problem Understanding

An organization has multiple network links connecting different branches. The administrator needs to continuously monitor bandwidth utilization and identify overloaded links.

The algorithm should:

- Store network links in an array/list.
- Periodically collect bandwidth utilization.
- Use a loop to monitor all links.
- Store utilization values.
- Detect utilization greater than **80%**.
- Recommend switching traffic to a backup link.
- Continue monitoring periodically.

Algorithm Design

BEGIN

SET links = ["Link1", "Link2", "Link3", "Link4", "Link5"]

SET utilization = [0, 0, 0, 0, 0]

SET monitoring_interval = 10

WHILE TRUE DO

 DISPLAY "BANDWIDTH MONITORING STARTED"

 FOR i = 0 TO LENGTH(links) - 1 DO

 DISPLAY "Enter bandwidth utilization for ", links[i], ": "

 INPUT utilization[i]

 IF utilization[i] < 0 OR utilization[i] > 100 THEN

 DISPLAY "Invalid utilization value"

 CONTINUE

 END IF

 IF utilization[i] > 80 THEN

 DISPLAY links[i], " is OVERLOADED"

 DISPLAY "Utilization: ", utilization[i], "%"

 DISPLAY "Recommendation: Switch traffic to backup link"

```

ELSE

    DISPLAY links[i], " is operating normally"
    DISPLAY "Utilization: ", utilization[i], "%"

END IF

END FOR

DISPLAY "CURRENT BANDWIDTH SUMMARY"

FOR i = 0 TO LENGTH(links) - 1 DO
    DISPLAY links[i], ": ", utilization[i], "%"
END FOR

WAIT monitoring_interval seconds

END WHILE

END

```

Problem Decomposition

The problem can be divided into four major tasks:

1. Store network link information.
2. Collect bandwidth utilization.
3. Compare utilization with the 80% threshold.
4. Recommend an alternate link when overload occurs.

The array utilization[] stores the current bandwidth usage of each link.

Application of Constraints

The important constraint is:

Bandwidth utilization > 80% → Link is overloaded

For example:

```

Link1 = 45%
Link2 = 72%
Link3 = 86%
Link4 = 60%
Link5 = 91%

```

The algorithm identifies:

Link3 → OVERLOADED

Link5 → OVERLOADED

It then recommends switching traffic from those links to backup links.

Use of Control Structures

The algorithm uses a **WHILE loop** for continuous monitoring.

Inside the WHILE loop, a **FOR loop** checks every network link.

An IF-ELSE condition determines whether a link is overloaded.

```
IF utilization > 80 THEN
    DISPLAY "Switch traffic to backup"
ELSE
    DISPLAY "Link operating normally"
END IF
```

This makes the monitoring process automatic and continuous.

Logical Reasoning & Strategy

Continuous monitoring allows the administrator to identify congestion before the network becomes severely overloaded.

For example:

Link1 → 50%

Link2 → 65%

Link3 → 84%

Since Link3 exceeds 80%, the algorithm immediately recommends moving some traffic to a backup link.

This can reduce:

- Network congestion
- Packet loss
- Delay
- Application slowdown
- Connection failures

Completeness

The algorithm handles:

- Multiple network links.
- Continuous monitoring.
- Bandwidth storage.
- Invalid utilization values.
- Overload detection.
- Normal link detection.
- Backup-link recommendation.
- Periodic monitoring.

How the Algorithm Supports Efficient Bandwidth Management

The algorithm helps administrators make better use of available network resources.

When utilization is below 80%, the link can continue operating normally.

When utilization exceeds 80%, traffic can be redirected to another available link.

This provides **load balancing** and prevents a single link from becoming a bottleneck.

For example:

Before:

Main Link = 90%
Backup Link = 30%

After Traffic Switching:

Main Link = 65%
Backup Link = 55%

Traffic is distributed more efficiently, improving overall network performance.

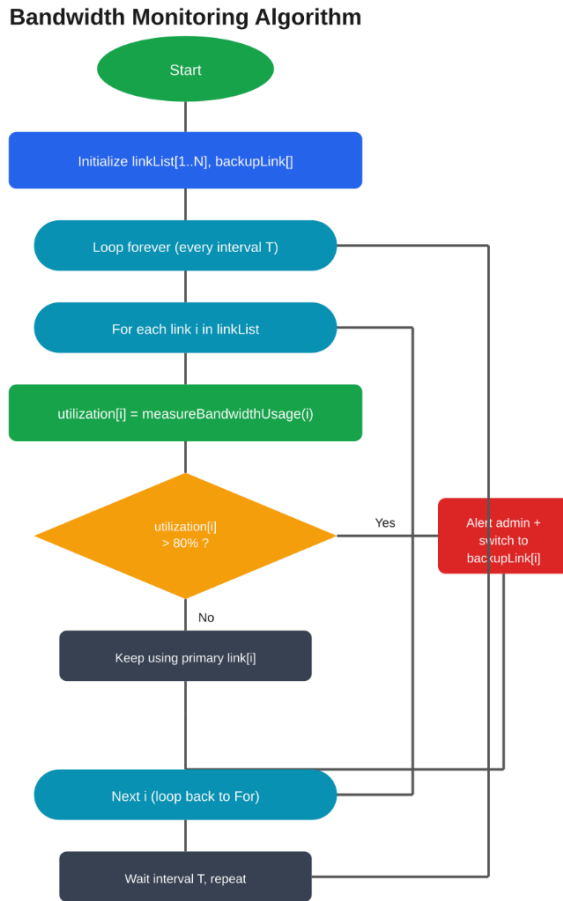


Figure 2: Flowchart — periodic per-link monitoring with automatic backup-link failover

3. Best-Path Selection Based on Overall Link Quality

Problem Understanding

The company has six branch offices connected through multiple network links. Each link has different:

- Bandwidth
- Delay
- Utilization
- Availability

These values change throughout the day.

Therefore, selecting a path only based on the shortest physical distance is not sufficient. The algorithm must dynamically select the path with the best overall quality.

The algorithm must:

- Periodically collect bandwidth.
- Collect delay.
- Collect utilization.
- Store values in arrays/lists.
- Calculate a quality score.
- Select the path with the highest score.
- Detect overloaded links.
- Detect failed links.
- Select alternate paths.
- Generate administrator alerts.
- Repeat the process continuously.

Problem Decomposition

The problem can be divided into the following steps:

1. Identify all available network paths.
2. Collect bandwidth information.
3. Collect delay information.
4. Collect utilization information.
5. Check link availability.
6. Calculate link quality.
7. Compare quality scores.
8. Select the best path.
9. Detect overloaded or failed links.
10. Select an alternate path when necessary.
11. Alert the administrator.
12. Repeat monitoring periodically.

Algorithm Design

A suitable link-quality score can be calculated using normalized values.

Higher bandwidth should increase the score.

Lower delay should increase the score.

Lower utilization should increase the score.

One possible formula is:

Where:

This gives higher scores to paths with high bandwidth, low delay, and low utilization.

Pseudocode

BEGIN

SET paths = ["Path1", "Path2", "Path3", "Path4", "Path5", "Path6"]

SET bandwidth[6]

SET delay[6]

SET utilization[6]

SET quality_score[6]

SET status[6]

SET monitoring_interval = 10

WHILE TRUE DO

 DISPLAY "NETWORK PATH MONITORING STARTED"

 max_bandwidth = 0

 max_delay = 0

 FOR i = 0 TO 5 DO

 DISPLAY "Enter bandwidth for ", paths[i], ": "

 INPUT bandwidth[i]

 DISPLAY "Enter delay for ", paths[i], ": "

 INPUT delay[i]

 DISPLAY "Enter utilization for ", paths[i], ": "

 INPUT utilization[i]

 DISPLAY "Enter status for ", paths[i], " (UP/DOWN): "

 INPUT status[i]

 IF bandwidth[i] > max_bandwidth THEN

 max_bandwidth = bandwidth[i]

 END IF

 IF delay[i] > max_delay THEN

 max_delay = delay[i]

 END IF

 END FOR

 IF max_bandwidth <= 0 THEN

 DISPLAY "No valid network bandwidth available"

```
STOP
END IF
```

```
SET best_path = "NONE"
SET highest_score = -1
```

```
FOR i = 0 TO 5 DO
```

```
    IF status[i] = "DOWN" THEN
```

```
        quality_score[i] = 0
```

```
        DISPLAY paths[i], " has FAILED"
        DISPLAY "Alert: ", paths[i], " is unavailable"
```

```
    ELSE IF utilization[i] > 80 THEN
```

```
        quality_score[i] = 0
```

```
        DISPLAY paths[i], " is OVERLOADED"
        DISPLAY "Alert: High utilization detected"
```

```
    ELSE
```

```
        bandwidth_score =
            (bandwidth[i] / max_bandwidth) * 100
```

```
        delay_score =
            (1 - (delay[i] / max_delay)) * 100
```

```
        utilization_score =
            (1 - (utilization[i] / 100)) * 100
```

```
        quality_score[i] =
            (0.5 * bandwidth_score) +
            (0.3 * delay_score) +
            (0.2 * utilization_score)
```

```
        DISPLAY paths[i],
            " Quality Score = ",
            quality_score[i]
```

```
        IF quality_score[i] > highest_score THEN
```

```

        highest_score = quality_score[i]
        best_path = paths[i]

    END IF

END IF

END FOR

IF best_path = "NONE" THEN

    DISPLAY "ALERT: No available network path"
    DISPLAY "Network communication cannot be established"

ELSE

    DISPLAY "BEST PATH SELECTED: ", best_path
    DISPLAY "BEST PATH QUALITY SCORE: ", highest_score

END IF

DISPLAY "NETWORK PATH SUMMARY"

FOR i = 0 TO 5 DO

    DISPLAY paths[i],
        " | Bandwidth = ", bandwidth[i],
        " | Delay = ", delay[i],
        " | Utilization = ", utilization[i],
        " | Score = ", quality_score[i],
        " | Status = ", status[i]

END FOR

DISPLAY "Waiting for next monitoring cycle..."

WAIT monitoring_interval seconds

END WHILE

END

```

Application of Constraints

The algorithm considers all the important network constraints.

Bandwidth

A path with higher available bandwidth receives a higher bandwidth score.

For example:

Path A = 100 Mbps

Path B = 500 Mbps

Path B receives a better bandwidth score.

Delay

Lower delay is preferred because it improves communication responsiveness.

For example:

Path A = 20 ms

Path B = 100 ms

Path A receives a better delay score.

Utilization

A link with lower utilization has more available capacity.

For example:

Path A = 40%

Path B = 85%

Path A is preferred.

If utilization exceeds **80%**, the algorithm marks the path as overloaded and removes it from the candidate paths.

Failure Detection

If a link status becomes DOWN, its quality score is set to zero.

The algorithm automatically searches for another available path.

Logical Reasoning & Strategy

The algorithm does not select a route merely because it is geographically short.

Instead, it evaluates the **current quality of every path**.

For example:

Path	Bandwidth	Delay	Utilization	Status
------	-----------	-------	-------------	--------

Path 1	100 Mbps	20 ms	40%	UP
--------	----------	-------	-----	----

Path 2	500 Mbps	50 ms	50%	UP
--------	----------	-------	-----	----

Path 3	200 Mbps	15 ms	90%	UP
--------	----------	-------	-----	----

Path 4	300 Mbps	30 ms	45%	UP
--------	----------	-------	-----	----

Path 5	100 Mbps	80 ms	60%	UP
--------	----------	-------	-----	----

Path 6	400 Mbps	25 ms	30%	UP
--------	----------	-------	-----	----

Although Path 3 has a low delay, its utilization is **90%**, which exceeds the 80% threshold. Therefore, it should not be selected.

Path 6 may become the preferred path because it provides:

- High bandwidth
- Low delay
- Low utilization

This demonstrates why considering multiple metrics produces better routing decisions.

Use of Control Structures

The algorithm uses several control structures.

WHILE Loop

The WHILE loop continuously monitors the network.

```
WHILE TRUE DO
  monitor network
  calculate scores
  select path
  WAIT
END WHILE
```

FOR Loop

The FOR loop examines all six paths.

```
FOR i = 0 TO 5 DO
```

collect link information
END FOR

IF-ELSE Conditions

Conditions identify:

- Failed links
- Overloaded links
- Available links
- Best-quality paths

Solution Accuracy & Feasibility

The algorithm provides dynamic routing decisions based on changing network conditions.

Suppose the current best path is:

Path 6

Later, its utilization increases from:

30% → 92%

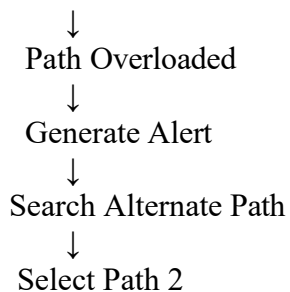
The next monitoring cycle identifies Path 6 as overloaded.

The algorithm then excludes it and selects the next best available path.

For example:

Previous Best Path → Path 6

Path 6 Utilization → 92%



This provides automatic adaptation to changing conditions.

Creativity & Innovation

The algorithm can be extended to use additional parameters such as:

- Packet loss
- Jitter
- Error rate
- Reliability
- Cost
- Security level

A more advanced scoring model could therefore be:

where the weights can be adjusted according to the organization's requirements.

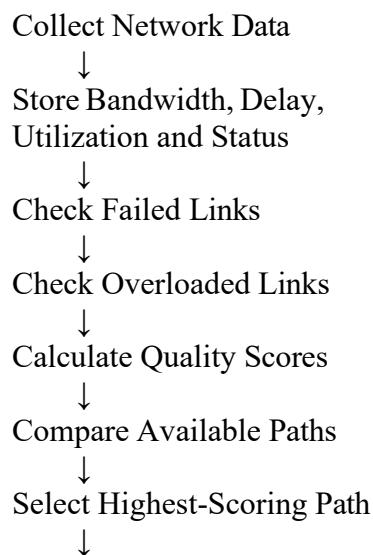
For a real-time video application, delay and jitter could receive higher weights. For large file transfers, bandwidth could receive a higher weight.

This makes the algorithm adaptable to different network applications.

Clarity of Presentation

Metric	Preferred Condition	Effect on Score
Bandwidth	Higher	Score increases
Delay	Lower	Score increases
Utilization	Lower	Score increases
Link Status	UP	Path considered
Link Status	DOWN	Path rejected
Utilization > 80%	Overloaded	Path rejected
Quality Score	Highest	Best path selected

Overall Process



Generate Alerts if Required



Wait for Next Monitoring Cycle



Repeat

Completeness

The algorithm handles all major cases:

- Normal network operation.
- High bandwidth utilization.
- Link overload.
- Link failure.
- Alternate path selection.
- Continuous monitoring.
- Quality-score calculation.
- Administrator alerts.
- No available path condition.
- Changing network conditions.

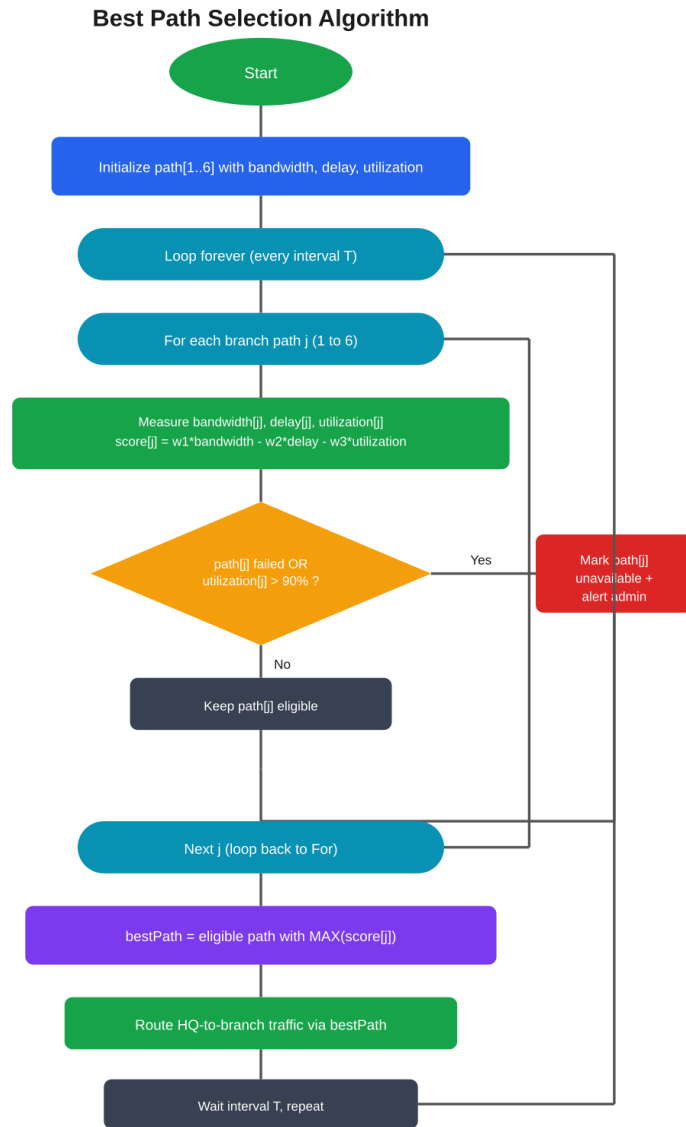


Figure 3: Flowchart — periodic scoring of all paths with automatic failover on overload/failure